Lab-9 # Create and deploy container images.

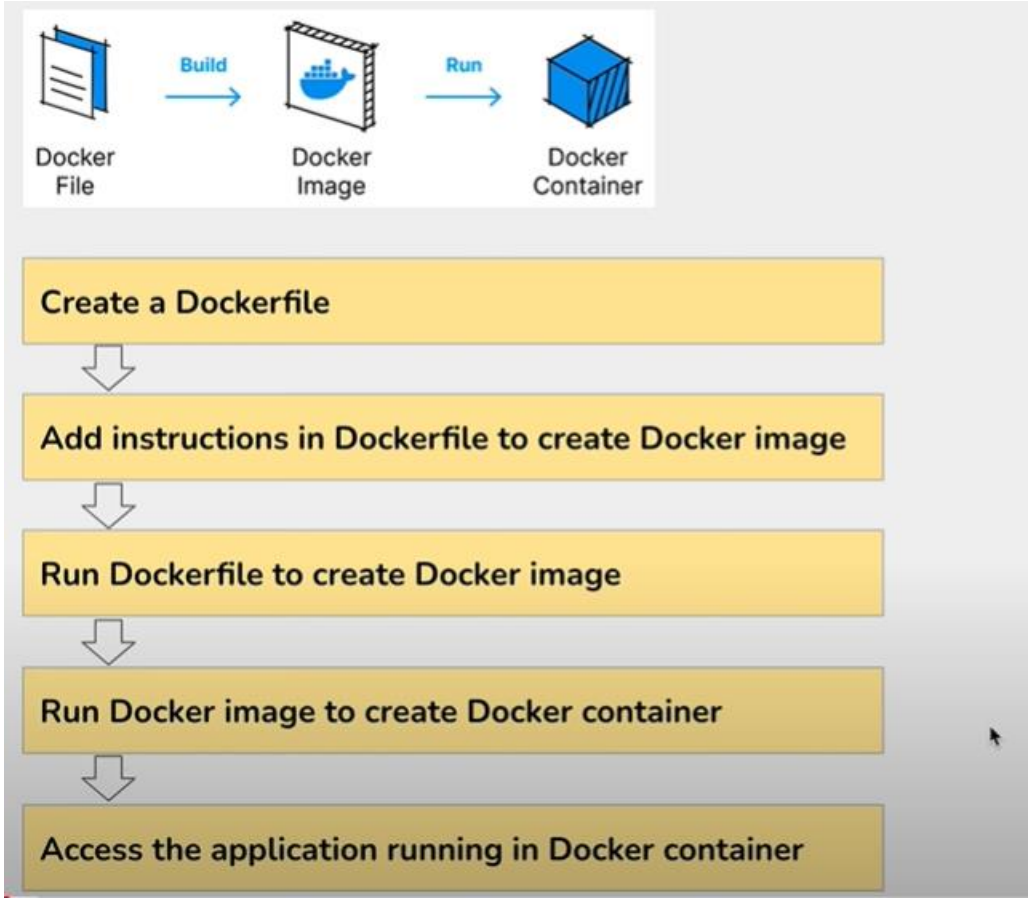
A. Create a container image for deployment to Azure Container Instances

❖ **Docker**:

✓ You need Docker installed locally. Docker provides packages that configure the Docker environment on macOS, Windows, and Linux.

✓ Install the docker and also install the Node.js

❖ **Create and deploy container images**



✓ Use Git to clone the sample application's repository*: use this link and download the project.*

git clone **https://github.com/Azure-Samples/aci-helloworld.git**

✓ Open the node.js project in VS code.

✓ The docker file is already created in github project.

✓ Dockerfile.

```
FROM node:8.9.3-alpine

RUN mkdir -p /usr/src/app

COPY ./app/* /usr/src/app/

WORKDIR /usr/src/app

RUN npm install

CMD node /usr/src/app/index.js
```

- ✓ If you want to create your own docker file please make sure the file is in the same directory of the Node.js project.

- ❖ **Commands**

- ✓ Use the docker build command to create the container image and tag it as *aci-tutorial-app*:

docker build ./aci-helloworld -t aci-tutorial-app

Output Copy

```
Sending build context to Docker daemon  119.3kB
Step 1/6 : FROM node:8.9.3-alpine
8.9.3-alpine: Pulling from library/node
88286f41530e: Pull complete
84f3a4bf8410: Pull complete
d0d9b2214720: Pull complete
Digest: sha256:c73277ccc763752b42bb2400d1aaecb4e3d32e3a9dbedd0e49885c71bea07354
Status: Downloaded newer image for node:8.9.3-alpine
 ---> 90f5ee24bee2
...
Step 6/6 : CMD node /usr/src/app/index.js
 ---> Running in f4a1ea099eec
 ---> 6edad76d09e9
Removing intermediate container f4a1ea099eec
Successfully built 6edad76d09e9
Successfully tagged aci-tutorial-app:latest
```

- ✓ Use the docker images command to see the built image:

docker images

Output

```
REPOSITORY          TAG       IMAGE ID        CREATED           SIZE
aci-tutorial-app    latest    5c745774dfa9    39 seconds ago    68.1 MB
```

- ✓ **Run the container locally:** Before you deploy the container to Azure Container Instances, use docker run to run it locally and confirm that it works. The -d switch lets the container run in the background, while -p allows you to map an arbitrary port on your computer to port 80 in the container.

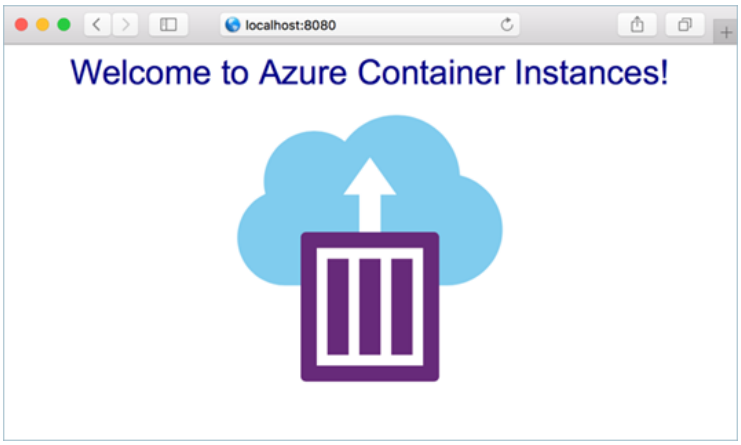
docker run -d -p 8080:80 aci-tutorial-app

- ✓ Output from the docker run command displays the running container's ID if the command was successful:

Bash

```
docker run -d -p 8080:80 aci-tutorial-app
```output
a2e3e4435db58ab0c664ce521854c2e1a1bda88c9cf2fcff46aedf48df86cccf
```

- ✓ Now, navigate to http://localhost:8080 in your browser to confirm that the container is running.



Website: https://learn.microsoft.com/en-us/azure/container-instances/container-instances-tutorial-prepare-app
```

B. Create an Azure container registry and push a container image

Azure Container Registry is your private Docker registry in Azure.

- ✓ Create an Azure Container Registry instance with the Azure CLI
- ✓ Tag a container image for your Azure container registry
- ✓ Upload the image to your registry
- ✓ Firstly, open the Command prompt and login the azure account

Using **az login**

❖ **Create Azure container registry:** Before you create your container registry, you need a *resource group* to deploy it to. A resource group is a logical collection into which all Azure resources are deployed and managed.

✓ Create a resource group with the az group create command. In the following example, a resource group named *myResourceGroup* is created in the *eastus* region:

az group create --name myResourceGroup --location eastus

✓ Once you've created the resource group, create an Azure container registry with the az acr create command. The container registry name must be unique within Azure, and contain 5-50 alphanumeric characters. Replace <acrName> with a unique name for your registry:

az acr create --resource-group myResourceGroup --name <acrName> --sku Basic

✓ ***demo example to create the container registery***

```
C:\Users\Pc>az acr create --resource-group myResourceGroup --name Containermy --sku Basic
argument error: Registry name must use only lowercase.

C:\Users\Pc>az acr create --resource-group myResourceGroup --name containermy --sku Basic
(AlreadyInUse) The registry DNS name containermy.azurecr.io is already in use. You can check if the name is already clai
med using following API: https://docs.microsoft.com/en-us/rest/api/containerregistry/registries/checknameavailability
Code: AlreadyInUse
Message: The registry DNS name containermy.azurecr.io is already in use. You can check if the name is already claimed us
ing following API: https://docs.microsoft.com/en-us/rest/api/containerregistry/registries/checknameavailability

C:\Users\Pc>az acr create --resource-group myResourceGroup --name name --sku Basic
(ResourceNameInvalid) Invalid resource name: 'name'. Resource names may contain alpha numeric characters only and must b
e between 5 and 50 characters.. For more information, please refer resource name requirements at: https://docs.microsoft
.com/en-us/rest/api/containerregistry/
Code: ResourceNameInvalid
Message: Invalid resource name: 'name'. Resource names may contain alpha numeric characters only and must be between 5 a
nd 50 characters.. For more information, please refer resource name requirements at: https://docs.microsoft.com/en-us/re
st/api/containerregistry/

C:\Users\Pc>az acr create --resource-group myResourceGroup --name acrnme --sku Basic
{
  "adminUserEnabled": false,
  "anonymousPullEnabled": false,
  "creationDate": "2024-05-02T05:47:23.729619+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
```

- **Log in to container registry:** You must log in to your Azure Container Registry instance before pushing images to it. Use the az acr login command to complete the operation. You must provide the unique name you chose for the container registry when you created it.

az acr login --name <acrName>

The command returns Login Succeeded once completed

```
C:\Users\Pc>az acr login --name acrnme
Login Succeeded
```

- **Tag container image:** To push a container image to a private registry like Azure Container Registry, you must first tag the image with the full name of the registry's login server.

- First, get the full login server name for your Azure container registry. Run the following az acr show command, and replace <acrName> with the name of registry you just created:

az acr show --name <acrName> --query loginServer --output table

```
C:\Users\Pc>az acr show --name acrnme --query loginServer --output table
Result
----------------
acrnme.azurecr.io
```

- ✓ Now, display the list of your local images with the docker images command: (run on vs terminal)

```
PS E:\LAB DATA\Labss\Cloud Computing\labs cloud sp24\project-acihello> docker images
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
aci-tutorial-app    latest    98e99cd9ece2   36 minutes ago   72.5MB
PS E:\LAB DATA\Labss\Cloud Computing\labs cloud sp24\project-acihello>
```

- ✓ Tag the *aci-tutorial-app* image with the login server of your container registry. Also, add the :v1 tag to the end of the image name to indicate the image version number. Replace <acrLoginServer> with the result of the az acr show command you executed earlier.

docker tag aci-tutorial-app <acrLoginServer>/aci-tutorial-app:v1

```
C:\Users\Pc>docker tag aci-tutorial-app acrnme.azurecr.io/aci-tutorial-app:v1
```

- ✓ Run docker images again to verify the tagging operation:

```
REPOSITORY                          TAG       IMAGE ID       CREATED          SIZE
acrnme.azurecr.io/aci-tutorial-app  v1        98e99cd9ece2   40 minutes ago   72.5MB
aci-tutorial-app                    latest    98e99cd9ece2   40 minutes ago   72.5MB
```

- ❖ **Push image to Azure Container Registry**
- ✓ Now that you've tagged the *aci-tutorial-app* image with the full login server name of your private registry, you can push the image to the registry with the docker push command. Replace <acrLoginServer> with the full login server name you obtained in the earlier step.
- ✓ Now use VS Code terminal

docker push <acrLoginServer>/aci-tutorial-app:v1

```
PS E:\LAB DATA\Labss\Cloud Computing\labs cloud sp24\project-acihello> docker push acrnme.azurecr.io/aci-tutorial-app:v1
The push refers to repository [acrnme.azurecr.io/aci-tutorial-app]
aacc34f8c69a: Pushing [==================================================>]  6.004MB
5f70bf18a086: Pushing  1.024kB
db5b8d7fdf63: Pushed
9fbec0cfb2ec: Pushed
1dfbdf308b77: Preparing
2ec940494cc0: Waiting
6dfaec39e726: Waiting
```

❖ **List images in Azure Container Registry**

✓ To verify that the image you just pushed is indeed in your Azure container registry, list the images in your registry with the az acr repository list command. Replace <acrName> with the name of your container registry.

✓ Using azure CLI

az acr repository list --name <acrName> --output table

```
C:\Users\Pc>az acr repository list --name acrnme --output table
Result
----------------
aci-tutorial-app
```

✓ To see the *tags* for a specific image, use the az acr repository show-tags command.

az acr repository show-tags --name <acrName> --repository aci-tutorial-app --output table

```
C:\Users\Pc>az acr repository show-tags --name acrnme --repository aci-tutorial-app --output table
Result
--------
v1
```

Website: ***https://learn.microsoft.com/en-us/azure/container-instances/container-instances-tutorial-prepare-acr***

C. Deploy a container application to Azure Container Instances

- ✓ Deploy the container from Azure Container Registry to Azure Container Instances
- ✓ View the running application in the browser
- ✓ Display the container's logs

❖ **Deploy the container using the Azure CLI**

- ✓ When you deploy an image that's hosted in a **private Azure container registry** like the one created in the ***Create an Azure container registry and push a container image.***
- ✓ A best practice for many scenarios is to create and configure a Microsoft Entra service principal with *pull* permissions to your registry. See Authenticate with Azure Container Registry from Azure Container Instances for sample scripts to create a service principal with the necessary permissions. Take note of the *service principal ID* and *service principal password*. You use these credentials to access the registry when you deploy the container.
- ✓ Here is the command

az ad sp create-for-rbac --name <service_principal_name> --role Contributor --scopes /subscriptions/<subscription_id>

```
C:\Users\Pc>az ad sp create-for-rbac --name service_principal_name --role Contributor --scopes /subscriptions/4c9f7251-f
e36-4db0-a50f-a3a54128e7cc
Creating 'Contributor' role assignment under scope '/subscriptions/4c9f7251-fe36-4db0-a50f-a3a54128e7cc'
The output includes credentials that you must protect. Be sure that you do not include these credentials in your code or
 check the credentials into your source control. For more information, see https://aka.ms/azadsp-cli
{
  "appId": "6c566f31-7cdd-4249-8832-e667f3ff6a06",
  "displayName": "service_principal_name",
  "password": "FG-8Q~DxkQVey9~6RBs5oXqmx0vULE6oY0O_HcmJ",
  "tenant": "1511ab2e-502b-4e2d-bd68-f679f549b5a2"
```

- ✓ You also need the full name of the container registry login server (replace <acrName> with the name of your registry):

az acr show --name <acrName> --query loginServer

```
C:\Users\Pc>az acr show --name acrnme --query loginServer
"acrnme.azurecr.io"
```

❖ **Deploy container**

- ✓ Now, use the az container create command to deploy the container.

  Replace <acrLoginServer> with the value you obtained from the previous command.

  Replace <service-principal-ID> and <service-principal-password> with the service principal

ID and password that you created to access the registry. Replace <aciDnsLabel> with a desired DNS name.

az container create --resource-group myResourceGroup --name aci-tutorial-app --image <acrLoginServer>/aci-tutorial-app:v1 --cpu 1 --memory 1 --registry-login-server <acrLoginServer> --registry-username <service-principal-ID> --registry-password <service-principal-password> --ip-address Public --dns-name-label <aciDnsLabel> --ports 80

```
C:\Users\Pc>az container create --resource-group myResourceGroup --name aci-tutorial-app --image acrnme.azurecr.io/aci-tutorial-app:v1 --cpu 1 --memory 1 --registry-login-server acrnme.azure
cr.io --registry-username 6c566f31-7cdd-4249-8832-e667f3ff6a06 --registry-password FG-8Q~DxkQVey9~6RBs5oXqmx0vULE6oY0O_HcmJ --ip-address Public --dns-name-label acrnme --ports 80
{
  "confidentialComputeProperties": null,
  "containers": [
    {
      "command": null,
      "environmentVariables": [],
      "image": "acrnme.azurecr.io/aci-tutorial-app:v1",
      "instanceView": {
```

❖ **Verify deployment progress**

✓ To view the state of the deployment, use az container show:

az container show --resource-group myResourceGroup --name aci-tutorial-app --query instanceView.state

✓ Repeat the az container show command until the state changes from *Pending* to *Running*, which should take under a minute. When the container is *Running*, proceed to the next step.

```
C:\Users\Pc>
C:\Users\Pc>az container show --resource-group myResourceGroup --name aci-tutorial-app --query instanceView.state
"Running"
```

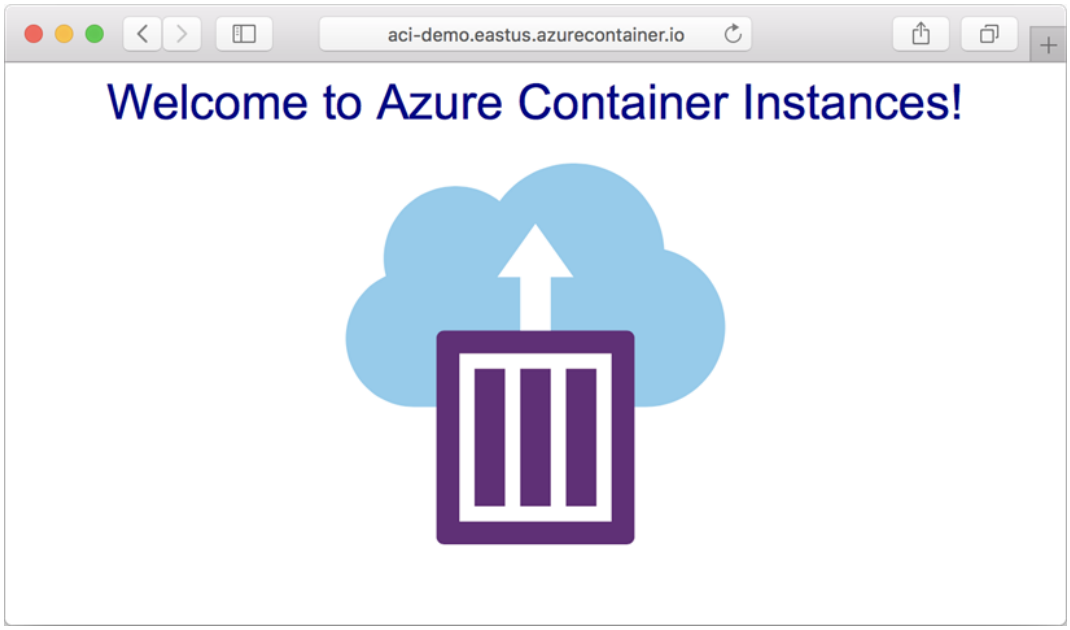
❖ **View the application and container logs**

✓ Once the deployment succeeds, display the container's fully qualified domain name (FQDN) with the az container show command:

az container show --resource-group myResourceGroup --name aci-tutorial-app --query ipAddress.fqdn

```
C:\Users\Pc>az container show --resource-group myResourceGroup --name aci-tutorial-app --query ipAddress.fqdn
"acrnme.eastus.azurecontainer.io"
```

✓ To see the running application, navigate to the displayed DNS name in your favorite browser:



❖ **You can also view the log output of the container:**

az container logs --resource-group myResourceGroup --name aci-tutorial-app

```
C:\Users\Pc>az container logs --resource-group myResourceGroup --name aci-tutorial-app
listening on port 80
::ffff:10.92.0.9 - - [02/May/2024:06:31:43 +0000] "GET / HTTP/1.1" 200 1663 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.
36"
::ffff:10.92.0.9 - - [02/May/2024:06:31:43 +0000] "GET /favicon.ico HTTP/1.1" 404 150 "http://acrnme.eastus.azurecontainer.io/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"
```

❖ **Clean up resources**

If you no longer need any of the resources you created in this tutorial series, you can execute the az group delete command to remove the resource group and all resources it contains. This command deletes the container registry you created, as well as the running container, and all related resources

az group delete --name myResourceGroup

Website: https://learn.microsoft.com/en-us/azure/container-instances/container-instances-tutorial-deploy-app

Lab task

Create and deploy the container image and attach all screen shots of your work.